

A framework for measuring data quality sensitivity in process discovery

Yannis Bertrand¹[0000–0002–6407–7221],
Martin Kabierski²[0000–0002–9852–7489], Jari Peeperkorn³[0000–0003–4644–4881],
and Seppe Vanden Broucke¹[0000–0002–8781–3906]

¹ Ghent University, Tweekerkenstraat 2, 9000 Ghent, Belgium

{yannis.bertrand, seppe.vandenbroucke}@ugent.be

² University of Vienna, Vienna, Austria

martin.kabierski@univie.ac.at

³ KU Leuven, Leuven, Belgium

jari.peeperkorn@kuleuven.be

Abstract. Process discovery techniques aim at generating models of business processes based on event logs. These techniques typically rely on assumptions about the event logs, such as the absence of concept drift and data quality issues that can impact process discovery results. However, this assumption rarely holds for real-life logs, which are often impacted by human or system errors. In this paper, we introduce a systematic framework for evaluating the impact of data quality issues on the outcomes of process discovery algorithms. In addition to this, we present an implementation of the framework as a flexible pipeline that can be used to evaluate the impact of various common data quality issues on user logs and is freely available online. The framework is validated by applying it to perform a sensitivity analysis of the impact of single data quality issues on well-known event logs. These two validation episodes demonstrate the soundness and usefulness of the framework in different applications.

Keywords: Process discovery · Data quality · Sensitivity analysis

1 Introduction

Process discovery is a family of process mining (PM) techniques that produce models of business processes based on data collected by information systems at runtime in so-called event logs [1]. Such models offer valuable insights into how processes are actually executed and support the identification of inefficiencies and opportunities for optimisation. To generate high-quality process models, process discovery techniques require that the event log is correct and complete. However, real-world process data are often affected by data quality issues (DQIs), such as missing events or incorrect activity labels, typically due to flaws in underlying information systems or human errors, which inadvertently negatively affect process discovery [7, 22, 23].

While it is widely acknowledged that DQIs negatively affect PM, little research has been conducted to systematically quantify this impact. Moreover, the broad

data analysis literature usually considers the impact of DQIs as task-specific. For instance, in machine learning, the severity of specific DQIs has been shown to vary with the specific downstream task [17]. While this is likely to be the case in PM as well, e.g., that the effect of different DQIs depends on the analysis task, such as process discovery, conformance checking, performance analysis, or predictive process monitoring, no previous research has investigated this issue. In light of this, two issues, i.e., the identification of the most relevant DQIs for an analysis task, and the sensitivity analysis of analysis tasks towards DQIs, are recognised as significant and timely challenges in the field of PM data quality management [23].

In this paper, we address these two challenges for the analysis task of process discovery. Specifically, we propose a framework for the assessment of how DQIs in event logs influence the models produced by process discovery algorithms. The framework is implemented as a modular pipeline that takes as input an event log, a list of DQIs, a set of discovery algorithms, and a set of quality metrics. Based thereon, it produces a sensitivity analysis, quantifying the impact of the individual DQIs on the outputs of the selected discovery techniques, in terms of the proposed quality metrics. Specifically, we demonstrate the framework’s utility by assessing the sensitivity of a set of well-established discovery algorithms to various DQIs, along several quality dimensions.

The remainder of the paper is structured as follows. Section 2 covers the relevant background of this paper. Then, Section 3 outlines our framework for the evaluation of DQI impact on process discovery. Section 4 then presents the evaluation of our framework, which is applied to perform a sensitivity analysis of discovery algorithms to common DQIs. Finally, Section 5 discusses our framework and some threats to its validity, before we conclude the paper in Section 6 with plans for future work.

2 Background

In this section, we introduce definitions used throughout out the remainder of the paper and relevant concepts from process discovery and data quality.

2.1 Event Logs

PM relies on the availability of a comprehensive event log that contains all events occurring within the analysed process over a specific time period. Formally, let $\mathcal{A}$ be the set of possible activities, $\mathcal{T}$ be the set of timestamps, and $\mathcal{C}$ be the set of case identifiers, i.e., a unique reference to an execution of the process. As an *event* we denote a tuple $e = (a \in \mathcal{A}, c \in \mathcal{C}, t \in \mathcal{T})$, i.e., a discrete occurrence of an activity at a specified point in time, matched to one specific case. With $e.a$, $e.c$, and $e.t$, we denote the activity, case identifiers, and timestamp associated with event e , respectively. Based thereon, a *trace* is a non-empty sequence $\xi = \langle e_1, e_2, \dots, e_n \rangle$ of events that share the same case identifier, i.e., $\forall i, j \in [1, 2, \dots, n] : e_i.c = e_j.c$ with $c \in \mathcal{C}$. Events in a trace are ordered by their timestamp, i.e., $\forall i, j \in [1, 2, \dots, n] : i < j \Leftrightarrow e_i.t < e_j.t$. Lastly, with $\mathcal{L}$ we denote the universe of possible event logs. An event log $L \in \mathcal{L}$ then is a finite set of traces, i.e., $L = \{\xi_1, \xi_2, \dots, \xi_k\}$.

2.2 Process Discovery and Model Evaluation

Process discovery describes a set of PM techniques aiming at automatically constructing models of business processes based on event logs [1]. Many process discovery techniques have been proposed so far, such as the α Miner [3], Inductive Miner (IM) [12], Declare Miner [14], and Split Miner [6]. These techniques try to identify recurrent patterns in an event log, e.g., two activities usually directly following each other, and combine these patterns to create process models that represent the behaviour present in the log as closely as possible. Most discovery algorithms to date generate Petri nets, but other popular discovery formalisms include BPMN, Declare, process trees, and causal nets. For a more complete treatment, we redirect the reader to [5].

Various metrics exist to evaluate the quality of a given process model generated by a discovery algorithm. These metrics typically compare the model with the event log from which it was discovered, using so-called conformance checking techniques, such as token-based replay [20] or alignments [2], which assess whether the traces recorded in an event log comply with a given process model along different quality dimensions. The most common quality dimensions are [8]: *fitness*, which measures to what extent the model accepts the traces present in the event log; *precision*, which measures whether the model only accepts traces that are present in the log; *generalisation*, which evaluates whether the model accepts traces that are not present in the event log; *simplicity*, which measures how complex a process model is, e.g., in terms of the number of elements.

2.3 Data quality in process mining

DQIs are discrepancies between the recorded event log and the real executed process [23]. DQIs in PM typically involve errors, inconsistencies, or missing information in event logs. An initial classification of DQIs was proposed based on two dimensions: the type of issue (*incorrect*, *irrelevant*, *imprecise*, or *missing* data) and the event log component affected (e.g., *cases*, *events*, *attributes*, *activity names*, *timestamps*, or *resources*) [7]. According to this work, the most common DQIs are: missing events, incorrect timestamps, imprecise activity names, imprecise timestamps, and irrelevant events. Building on this classification, [22] presents a list of 11 event log imperfection patterns, which describe recurring real-world situations that give rise to co-occurring DQIs. For each of these patterns, its cause, an example, the associated DQI category, and practical guidance for its detection and resolution are discussed.

Recently, [23] identified key challenges for improving data quality in PM, including best practices to prevent DQIs and ways to involve domain experts. The framework presented in this paper relates to the following two challenges laid out therein:

- *C14*: “Which dimensions of data quality are particularly important for process mining? Are certain dimensions more important for specific subareas of process mining?”
- *C15*: “How can we best show how process-data quality issues may have impacted analytical outcomes?”

A sensitivity analysis conducted using the proposed framework enables the quantitative identification of those DQIs, that affect process discovery algorithms the most. This, in turn, informs analysts of the most critical DQIs to consider in event logs for process discovery. Moreover, it can be used to identify the impact of DQIs on different algorithms, which can also steer algorithm selection.

3 Framework

The primary research problem addressed in this work is evaluating the impact of DQIs on process discovery outcomes. Accordingly, our main contribution is the development of a configurable pipeline that facilitates such a sensitivity analysis, together with its implementation in Python⁴, which makes it easily reusable by other researchers. Our framework requires four inputs or parameters: i) an event log to be analysed; ii) a list of DQIs whose impact on the log is to be assessed, together with potential parameters (e.g., the percentage of events to tamper with); iii) a set of process discovery algorithms and their configuration settings; iv) a set of conformance checking metrics for evaluating the quality of the resulting process models.

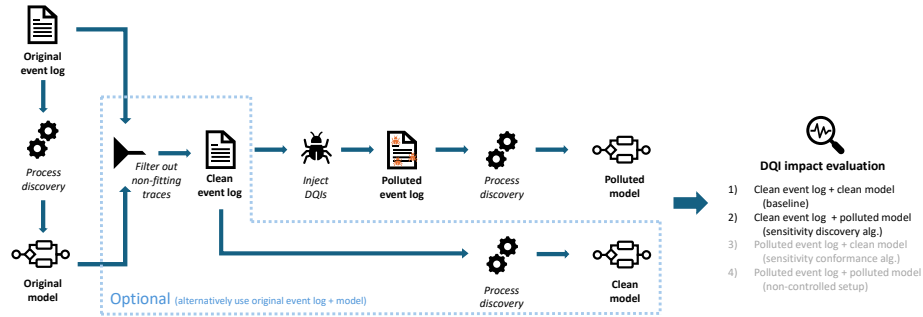


Fig. 1: Overview of our framework for the evaluation of DQI impact.

Using these inputs, the framework generates an evaluation of the impact of the specified DQIs on models discovered by the selected discovery algorithms, quantified as the difference in the chosen conformance checking metric. The following subsections detail each step of the framework, as illustrated in Figure 1.

3.1 Optional: Generate a clean event log

To ensure a clean starting point for controlled experiments, we optionally construct a clean event log that is free from pre-existing DQIs. This step is not strictly necessary if the original event log can reasonably be assumed to be clean. However, in many real-world cases, this assumption does not hold. To avoid uncontrolled influences from unknown or existing DQIs, we apply the following procedure to obtain a clean event log. This allows us to start from a well-defined state before introducing artificial DQIs.

⁴ <https://github.com/MartinKabierski/event-log-pollution-analysis>

1. Starting with the **original event log**, discover a process model that is *reasonably precise*. In the following, we refer to the model discovered in this step as the **original model**.
2. From the original event log, remove all traces which do not perfectly fit this model to create the so-called **clean log**.

The resulting clean log aligns with the behaviour described by the original model. While the model is discovered rather than normative, it offers a reliable proxy for filtering. Note that if a normative model of the process is available, it can constitute a better alternative for filtering out non-fitting traces. In our experiments, we used the Inductive Miner Infrequent algorithm (IMf) [12] with the infrequency threshold set to 0.2. We also verified manually that the discovered models were not overly permissive (e.g., flower models).

3.2 Inject data quality issues

Next, we systematically pollute the clean log to simulate the occurrence of DQIs. In this step, any type of event log DQI can be injected to reflect the exact issues present or presumed in the original log. Conceptually, we define a *polluter* as a function $\mathcal{P}: \mathcal{L} \rightarrow \mathcal{L}$, that systematically applies changes to an event log $L \in \mathcal{L}$ and outputs a **polluted log**. This is implemented with an abstract class `Polluter`, which implements a constructor taking the pollution level, i.e., the severity of the DQI, as a parameter, and a generic `pollute()` method, which is made specific by inheritance through child classes. Currently, our implementation supports the atomic DQIs, identified as most common according to [7]. Though we note that additional DQIs can be added easily. The currently supported DQIs are implemented as follows:

Missing Events. This DQI corresponds to the situation where some events that happened during the execution of a process are not present in the corresponding trace logged by the system. Concretely, this DQI is simulated as follows:

- Drop a certain percentage of event in L , selected randomly, implemented with class `DeleteActivityPolluter`.

Incorrect Timestamps. This DQI occurs when the timestamp associated with some events does not correspond to the time at which an event actually occurred, due to, e.g., delayed manual logging, which affects the ordering of events in affected traces. This DQI is simulated as follows:

- Add a random number of minutes of delay to the timestamp of a given percentage of the events in L , selected randomly. In our implementation, the number of minutes is sampled from a Gamma distribution with a user-specified average, which yields a skewed distribution with a minimum of 0 and a longer tail to the right, implemented with class `DelayedEventLoggingPolluter`

Imprecise Activity Names. Typically, this DQI entails multiple slightly different activities having the same label, or multiple variants of an activity that have the same label, e.g., handling a claim with a large amount and handling a claim with a low amount both being logged equally as “handle claim”. This DQI is simulated as follows:

- Specify a set of fine-grained activity labels and merge them into the same coarse-grained activity label, implemented with class `ImpreciseActivityPolluter`

Imprecise Timestamps. This DQI corresponds to the situation where the timestamps of events are not precise enough to derive the exact ordering of the events, i.e., some events have the same timestamp although they did not occur at the same time. This DQI is simulated as follows:

- Aggregate timestamps at a coarse granularity level (currently supported granularity levels: second, minute, 15 minutes, hour, day), implemented with class `AggregatedEventLoggingPolluter`

Irrelevant Events. This DQI occurs when some events pertain to activities that are outside the scope of the process to be analysed, i.e., they are alien to the process. This DQI is simulated as follows:

- Inject a certain number of new events with an “alien” activity label, e.g., if the process contains activities A, B, C, D, E , add an activity F , implemented with class `InsertAlienActivityPolluter`

3.3 Apply process discovery technique

In this stage, the process discovery algorithm specified by the analyst is applied to generate process models. To evaluate the sensitivity of the algorithm towards the specified DQI, discovery is applied to both the polluted event log and the clean event log, resulting in a **polluted model** and a **clean model**, respectively. Note that the framework is independent of a specific discovery algorithm. As such, it can be instantiated with any desired discovery algorithm.

3.4 Evaluate process models

The final step is to evaluate the discovered models using the specified conformance checking metrics. As mentioned above, there are multiple dimensions along which the quality of a model can be measured, with multiple specific algorithms implementing each. For instance, fitness, precision, and generalization could be measured by token-based replay [20]. Likewise, using alignment-based conformance checking, fitness or precision could be measured [4]. In any case, the framework is flexible and can be instantiated with any user-chosen evaluation metric, even going beyond the four specified evaluation dimensions.

At this stage, we have two logs, the clean log and the polluted log, and two models, the discovered clean model and the polluted model. Theoretically, this yields four log-model combinations for which conformance measures could be compared, which can yield complementary information, as is depicted in Figure 1. In particular, the computation of conformance measures between the clean event log and the polluted model aims to isolate the effect of DQIs on the discovery step, as the quality of the discovered (polluted) model is evaluated against clean data. Therefore, this combination is the focus of the experiments in the main body of this work.

4 Experimental evaluation

In this section, we demonstrate the usefulness of our framework by conducting a sensitivity analysis of common DQIs on several discovery algorithms. The framework and scripts are available online, together with the full results.

4.1 Experimental setup

For the sensitivity analysis, we applied our pipeline iteratively with the following inputs:

Original event logs. We used Road Traffic Fines Management [13], Sepsis Cases [15], Helpdesk [18], and Hospital billing [16]. As all logs are assumed to contain DQIs, we discovered the original model using IMf (with threshold 0.2), and filtered out all non-fitting traces to obtain the clean log.

DQIs. To each clean event log, we applied each DQI using the corresponding polluter as follows:

- Missing events (I2): applied using `DeleteActivityPolluter`
- Incorrect timestamps (I16): applied using `DelayedEventLoggingPolluter`, with average delay set as the average time gap between consecutive events (for each log), ensuring that the delays can affect the order of events without yielding exaggerated shifts.
- Imprecise activity name (I22): applied using `ImpreciseActivityPolluter`. Note that this DQI could only be applied to the Sepsis log, as it is the only log that explicitly contains multiple variants of the same activity. There, we translated *Release A*, *Release B*, *Release C*, *Release D* and *Release E* into a coarser activity label *release*
- Imprecise timestamps (I23): applied using `AggregatedEventLoggingPolluter`, with two different granularity levels for the coarser-grained timestamps: *hourly* and *daily* levels
- Irrelevant events (I27): applied using `InsertAlienActivityPolluter`. In our experiments, we generate several alien activity labels to insert in each log equal to the square root of the number of different activities registered in the log, e.g., if the log contains 16 different activities, 4 new alien activities are considered.

We applied each pollution pattern to 10%-90% of the events in the log, in increments of 10%. Note that the `ImpreciseActivityPolluter` could only be applied to the whole log.

Discovery algorithms. For discovery algorithms to evaluate, we selected the α -miner [3], Inductive Miner with noise thresholds of 0.0, and 0.2 (*IM*) [12], and the ILP miner with noise thresholds of 1.0, and 0.8 (*ILP*) [10].

Evaluation metrics. As evaluation metrics, we measured fitness and precision based on token-based replay [4].

4.2 Results

To visualise performance, we plotted fitness and precision scores across increasing levels of DQI pollution (in Figures 2 and 3, respectively), showing how the model

quality of different discovery algorithms degrades as event logs become increasingly polluted. As mentioned earlier, we compared the clean log with the polluted model, as this comparison isolates the effect of DQIs on the discovery algorithm. Due to space limitations, we report the average scores over all logs, although the full results per event log are available online in our GitHub repository.

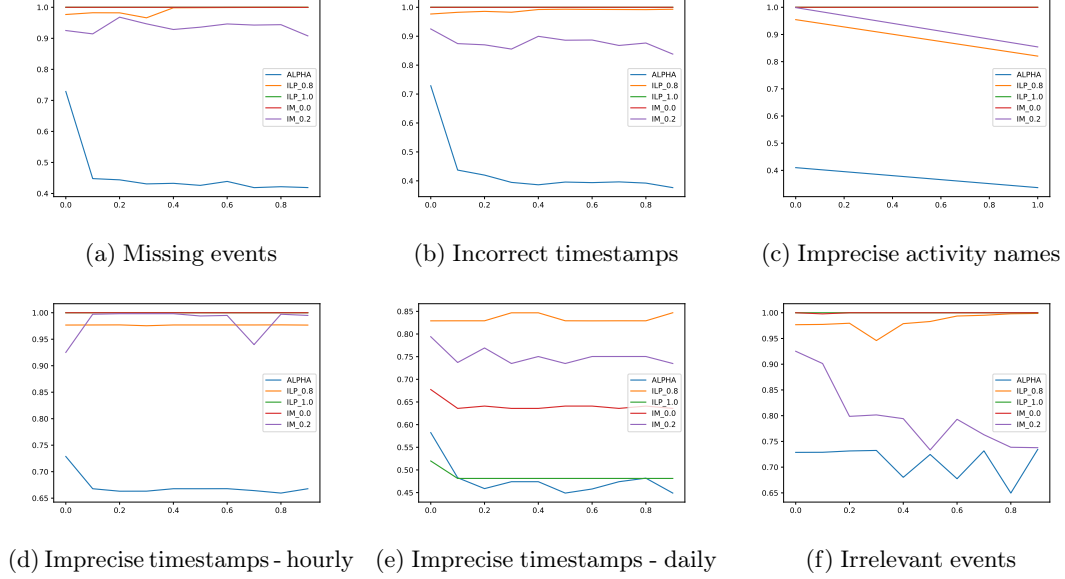


Fig. 2: Token-based fitness on the clean log, and discovered polluted models at different event log pollution levels (for each pattern).

Figure 2 shows that fitness remains relatively stable across most DQIs, though some algorithms demonstrate higher robustness. In particular, ILP with a 0.8 threshold and IM without filtering exhibit consistently high fitness, while α -miner shows greater sensitivity to noise. This stability is expected, since using the clean log makes sure a polluted model is not disadvantaged for not being able to replay noisy behaviour, i.e., additional behaviour that was artificially injected. As long as most control-flow constructs present in some (clean) traces are still captured by the discovered model, fitness remains high.

That being said, the precision of the models seems to suffer more from DQIs. Here, in Figure 3, we observe a drop in the precision of the models discovered by all algorithms across most DQIs. Lower precision indicates that by adding DQIs, the discovered polluted model allows for more incorrect behaviour (according to the clean log). Interestingly, both ILP with a noise threshold of 0.8 and IM with a noise threshold of 0.2 discovery techniques generally outperform other algorithms.

Different DQI types also exhibit varied impacts. Timestamp-related DQIs (especially imprecise activity names) exhibit minimal impact on fitness and precision. In contrast, direct modifications of the control-flow, such as event deletion (miss-

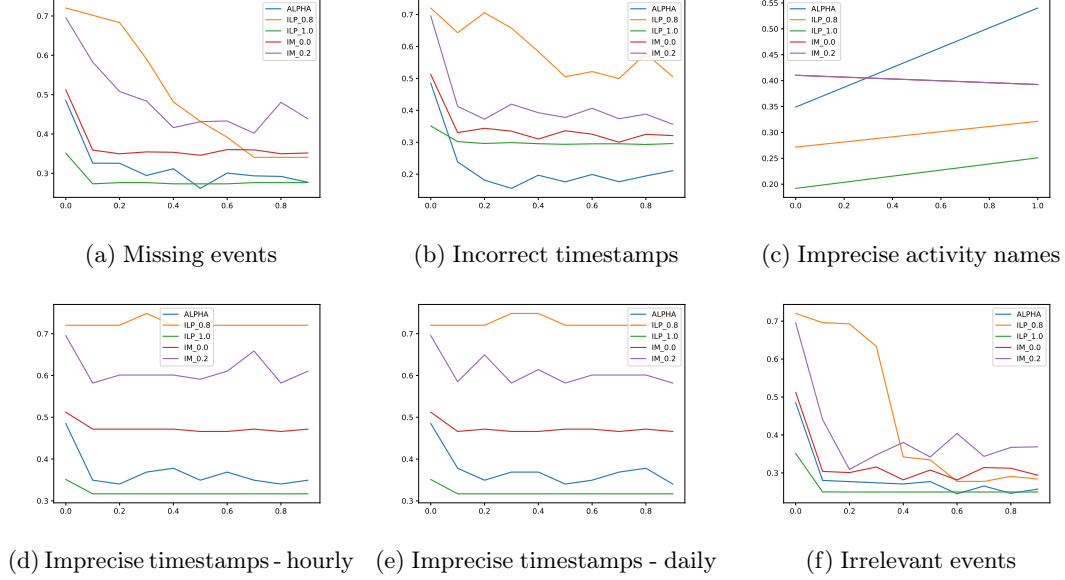


Fig. 3: Token-based precision on the clean log, and discovered polluted models at different event log pollution levels (for each pattern).

ing events) and insertion (irrelevant events), considerably degrade model quality. Imprecise activity names show a nuanced effect, sometimes decreasing fitness while increasing precision, possibly by simplifying (and therefore restricting) the model. Another notable pattern is the sharp performance drop when small amounts of noise (e.g., 10%) are introduced, which is followed by a plateau. This suggests that the presence of DQIs, even at small magnitudes, can have a severe impact on discovery quality, materially influencing the suitability of a discovery algorithm.

5 Discussion

The first observation is that a holistic interpretation of the model quality metrics is difficult, since these only evaluate whether a model closely matches the traces present in an event log. Because of this, for some metrics, the polluted model tested against the clean event log may yield similar or better quality than the clean model tested against the clean event log, e.g., if some infrequent variants are deleted in the polluted log. For this reason, it would be necessary to jointly evaluate multiple metrics together. Such an integrated sensitivity analysis could provide more nuanced evidence about the impact of DQIs on the performance of discovery algorithms. In light of this, one particular approach that could shed new light on the quality of discovered models is stochastic conformance checking [19], which may assess the impact of DQIs on activity label distributions. Moreover, it could be noted in Figure 1 that, for example, the polluted event log - clean model combination could also be

used to evaluate the sensitivity of the conformance checking algorithm’s outputs. Our online repository contains experiments with these other combinations as well.

Next, the output of our framework is still to some extent exploratory, i.e., we do not compute statistical tests to compare the performance of discovery algorithms more formally. While this would be relevant for large-scale experiments to benchmark the sensitivity of discovery algorithms to DQIs, the computation of such metrics requires the availability of a large number of event logs. In future work, this could be achieved by sampling from the clean event log to generate more event logs to be polluted and comparing them.

Furthermore, some of the DQIs we implemented and experimented with can be parameterised (e.g., `DelayedEventLogging` relies on an average delay), and their effect on process discovery is likely to depend on the values of their parameters. In our experiments, we addressed this issue by using realistic and log-dependent heuristics to determine the value of such parameters. However, in real-life use cases, domain knowledge could be used to help determine suitable values for each parameter. Additionally, it has been shown that in real-life, DQIs do typically not occur isolated, but rather co-occur [9, 22]. While we here only evaluated the sensitivity towards isolated, atomic occurrences of DQIs, assessing the impact that co-occurring DQIs have on model quality is left as future work.

Lastly, in our experiment, we could not assume the input logs to be clean. Due to this, an original model was discovered using the Inductive Miner Infrequent. The so obtained model may not necessarily be the *ground truth*. The discovered models display hidden transitions and skip-mechanisms, which could indicate that the discovered models are not entirely precise, and might accept too many variants. In future work, solutions to this issue could be to specify a precision threshold for the original model or include the technique used to discover the original model as a parameter for sensitivity analysis. Alternatively, the clean log could have been obtained using artificial (simulated) process data. While this would allow full control of the experiments and inclusion of a true ground truth model, for DQIs pertaining to e.g. timestamps, this would require a significant amount of choices or assumptions. Then, another option to obtain the original model could be to manually design a process model to match the original event log. That being said, manual modelling is a delicate task, which does not guarantee that the resulting model will resemble the ground truth model more closely.

When looking at the broader picture, this paper contributes to the current research on evaluating process discovery approaches, with DQIs being an important underlying factor affecting process discovery performance. A related perspective is offered in [24], which discusses the maturity stages of process discovery algorithms. Although the focus differs, it similarly investigates the interplay between event log quality and model quality, reinforcing the need to better understand how data quality impacts discovery results. Moreover, [21] recently proposed to evaluate process discovery techniques based on ground truth knowledge, which involves tampering with the process model generating the event log rather than the event log itself. To motivate their approach, they also invoke the concern that real-life event logs cannot be assumed to be of perfect quality. In this regard, the assessment

of log completeness [11] could be taken into account as a mediating factor between data quality and process discovery performance.

6 Conclusion

This paper presents two main contributions: a framework to evaluate the impact of DQIs on models generated by process discovery algorithms, and the implementation of the framework that can be used by other researchers, thereby addressing important challenges in PM data quality research. The effectiveness and flexibility of the framework are demonstrated in our experimental evaluation, where we conducted a sensitivity analysis of well-known process discovery techniques applied to publicly available event logs polluted with common DQIs, showing that the framework can be used for assessing the impact of various DQIs and for identifying process discovery algorithms most robust to them.

In future work, we plan to extend the implementation to support additional pollution patterns, and combinations thereof, as well as more process discovery algorithms, and conformance metrics. Then, we intend to perform a large-scale benchmark of process discovery techniques with many event logs to draw robust conclusions based on statistical tests. Another path for future research is the sensitivity evaluation of other types of analysis tasks, such as conformance checking, towards DQIs. Lastly, we intend to investigate the translation of defined pollution patterns into alternative definition schemes, such as the FLAWD language [9].

Acknowledgments. This work was supported in part by the Research Foundation Flanders (FWO) under Project 1294325N.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. van der Aalst, W.M.P.: Process mining: a 360 degree overview. In: *Process Mining Handbook*, pp. 3–34. Springer (2022)
2. Van der Aalst, W.M.P., Adriansyah, A., Van Dongen, B.F.: Replaying history on process models for conformance checking and performance analysis. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* **2**(2), 182–192 (2012)
3. Van der Aalst, W.M.P., Weijters, T., Maruster, L.: Workflow mining: Discovering process models from event logs. *IEEE TKDE* **16**(9), 1128–1142 (2004)
4. Adriansyah, A., Munoz-Gama, J., Carmona, J., van Dongen, B.F., van der Aalst, W.M.P.: Measuring precision of modeled behavior. In: *Information Systems and e-Business Management* **13**, 37–67 (2015). <https://doi.org/10.1007/s10257-014-0234-7>
5. Augusto, A., Conforti, R., Dumas, M., La Rosa, M., Maggi, F.M., Marrella, A., Mecella, M., Soo, A.: Automated discovery of process models from event logs: Review and benchmark. *IEEE transactions on knowledge and data engineering* **31**(4), 686–705 (2018)

6. Augusto, A., Conforti, R., Dumas, M., La Rosa, M., Polyvyanyy, A.: Split miner: automated discovery of accurate and simple business process models from event logs. *Knowledge and Information Systems* **59**, 251–284 (2019)
7. Bose, R.J.C., Mans, R.S., Van Der Aalst, W.M.: Wanna improve process mining results? In: 2013 IEEE CIDM. pp. 127–134. IEEE (2013)
8. Buijs, J.C., Van Dongen, B.F., van Der Aalst, W.M.P.: On the role of fitness, precision, generalization and simplicity in process discovery. In: CoopIS 2012, Rome, Italy, September 10–14, 2012. Proceedings, Part I. pp. 305–322. Springer (2012)
9. Comuzzi, M., Ko, J., Maggi, F.: A language to model and simulate data quality issues in process mining. *ACM Journal of Data and Information Quality* (2025)
10. van der Werf, J.M.E., van Dongen, B.F., Hurkens, C.A.J., Serebrenik, A.: Process discovery using integer linear programming. *fundamenta Informaticae* **94**(3–4), 387–412 (2009)
11. Kabierski, M., Richter, M., Weidlich, M.: Quantifying and relating the completeness and diversity of process representations using species estimation. *Information Systems* **130**, 102512 (2025)
12. Leemans, S.J., Fahland, D., Van Der Aalst, W.M.P.: Discovering block-structured process models from event logs containing infrequent behaviour. In: Business Process Management Workshops: BPM 2013 International Workshops, Beijing, China, August 26, 2013, Revised Papers 11. pp. 66–78. Springer (2014)
13. de Leoni, M.M., Mannhardt, F.: Road traffic fine management process (Feb 2015), https://data.4tu.nl/articles/_/12683249/1
14. Maggi, F.M., Mooij, A.J., van der Aalst, W.M.P.: User-guided discovery of declarative process models. In: 2011 IEEE symposium on computational intelligence and data mining (CIDM). pp. 192–199. IEEE (2011)
15. Mannhardt, F.: Sepsis cases - event log (Dec 2016), https://data.4tu.nl/articles/_/12707639/1
16. Mannhardt, F.: Hospital billing - event log (Aug 2017), https://data.4tu.nl/articles/_/12705113/1
17. Mohammed, S., Budach, L., Feuerpfeil, M., Ihde, N., Nathansen, A., Noack, N., Patzlaff, H., Naumann, F., Harmouch, H.: The effects of data quality on machine learning performance on tabular data. *Information Systems* **132**, 102549 (2025)
18. Polato, M.: Dataset belonging to the help desk log of an italian company (Jul 2017), https://data.4tu.nl/articles/_/12675977/1
19. Polyvyanyy, A., Kalenkova, A.: Conformance checking of partially matching processes: An entropy-based approach. *Information Systems* **106**, 101720 (2022)
20. Rozinat, A., Van der Aalst, W.M.P.: Conformance checking of processes based on monitoring real behavior. *Information Systems* **33**(1), 64–95 (2008)
21. Sommers, D., Sidorova, N., van Dongen, B.F.: A ground truth approach for assessing process mining techniques. *Process Science* **2**, 1 (2025). <https://doi.org/https://doi.org/10.1007/s44311-025-00006-8>
22. Suriadi, S., Andrews, R., ter Hofstede, A.H., Wynn, M.T.: Event log imperfection patterns for process mining: Towards a systematic approach to cleaning event logs. *Information systems* **64**, 132–150 (2017)
23. Ter Hofstede, A.H., Koschmider, A., Marrella, A., Andrews, R., Fischer, D.A., Sadeghianasl, S., Wynn, M.T., Comuzzi, M., De Weerd, J., Goel, K., et al.: Process-data quality: the true frontier of process mining. *ACM JDIQ* **15**(3), 1–21 (2023)
24. van der Werf, J.M.E., Polyvyanyy, A., van Wensveen, B.R., Brinkhuis, M., Reijers, H.A.: All that glitters is not gold: Four maturity stages of process discovery algorithms. *Information Systems* **114**, 102155 (2023)